SESSION 1 · USER TRACK

Using the Clever Cloud platform

An app, a database, a Kubernetes cluster — seen from the developer's side

Horacio Gonzalez · October 23, 2026 · 2 hours, mostly hands-on

“WE WON’T BE USING THE PLATFORM…”

Every ticket you’ll receive starts with a developer doing what we do today.

Creating an app. Deploying. Linking a database. Getting a kubeconfig.

WHY A USER TRAINING

Support without the user’s experience is guesswork

Qualify

Tell a platform problem from a usage mistake — the core job of L1/L2.

Reproduce

Redo what the user describes and ask the right questions.

Explain and escalate

Give the fix in the user’s words; write a useful escalation to L3.

Operate with empathy

Know which of your actions developers will feel.

AGENDA

Two hours, mostly hands-on

Time	Block	How
0:00	Why this session · setup check	Talk · Lab 0
0:10	Mental model: PaaS vs managed Kubernetes	Talk
0:25	A plain Python app	Hands-on · Lab 1
0:50	Add-ons: create, link, inject	Hands-on · Lab 2
1:10	A modern app with uv + PostgreSQL	Demo · Lab 3
1:20	Break	—
1:25	Clever Kubernetes Engine	Demo + hands-on · Lab 4
1:55	Wrap-up and what’s next	Talk

THE MENTAL MODEL

Where is the line between the platform and you?

Support’s first question, every time.

WHO DOES WHAT

PaaS application vs shared CKE cluster

Layer	PaaS application	CKE cluster
Who creates it	Each developer, per app	Platform team: 1–2 shared
Your code	You (Git push)	You (images, manifests)
Config and secrets	You (env vars)	You (ConfigMaps, Secrets)
Build and runtime	Platform	You (container images)
Scaling	Platform, within your bounds	You (replicas, node groups)
Load balancing, TLS	Platform	L4 LB: platform · TLS: you
Control plane	—	Platform
VMs, OS, hardware	Platform	Platform (heals failed VMs)
Data	Managed add-ons	Add-ons or your own volumes

RESOURCES

Everything lives in an organisation

Organisation
members · roles · billing

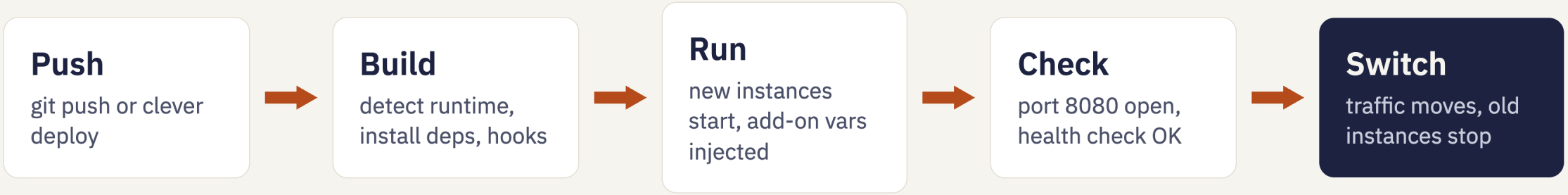
Applications
code + runtime + env vars + scaling

Add-ons
PostgreSQL, Redis, Cellar… linked to apps

CKE clusters
1–2 shared per organisation, run by the platform team

DEPLOYMENT LIFECYCLE

Every deployment builds new instances



Check fails? The deployment fails and the previous version keeps serving.

Build cache: a restart reuses the last build; a rebuild starts from scratch.

FOUR RULES OF THE GAME

What the platform expects from an app

Code in Git

One commit = one immutable deployment. Rollback = redeploy an older commit.

Config in env vars

Yours, the platform's (CC_*), the add-ons'. Change → restart.

State in add-ons

The instance disk is ephemeral: data goes to a database or object storage.

Listen on 0.0.0.0:8080

Logs go to stdout; the platform collects them.

LAB 1 · 25 MIN

Plain Python: three files, one setting

app.py — a small Flask app
requirements.txt — Flask, pg8000
.gitignore

No server config, no Dockerfile, no build script. The platform detects Python and serves the app with uWSGI behind Nginx.

The only mandatory setting:
CC_PYTHON_MODULE=app:app

```
git init && git add . && git commit -m "init"

clever create --type python \
--org {{ORG}} python-<name>

clever env set CC_PYTHON_MODULE app:app
clever env set CC_PYTHON_VERSION 3.13

clever deploy
clever domain
```

LAB 1 · READ THE DEPLOYMENT LOG

The log tells the story — read it first

Find the line for each step:

- the commit being deployed
- the Python version chosen
- dependencies installed from requirements.txt
- the build cache uploaded
- the check on port 8080
- the deployment success

```
clever activity
clever instances
clever logs --since 10m
clever accesslogs --since 10m

curl https://<app-domain>/
```

Blue/green: the new version starts before the old one stops




```
while true; do curl -s https://<app-domain>/ | grep -o '"version":"[^"]*"'; sleep 1; done
```

LAB 2 · 20 MIN

Add-ons: managed services next to your apps

Databases, caches, object storage, file systems, identity, analytics… Each provider has **plans** (size) and **zones**.

The platform provisions, monitors and backs them up. You choose the plan and own the data.

What's available on **your** installation? **clever addon providers**

```
clever addon providers
clever addon providers \
show postgresql-addon

clever addon create \
postgresql-addon pg-<name> \
--plan dev --org {{ORG}}
clever service link-addon pg-<name>
```

Linking injects the add-on’s variables when the app starts



A running instance doesn’t see them → restart. The #1 “my app doesn’t see the database” ticket.

LAB 3 · DEMO · 10 MIN

A modern app: Django, uv and PostgreSQL

github.com/CleverCloud/django-todo-list-example — watch, then answer:

1. Which file switches the platform to uv mode? Which variable is also required?
2. Why isn’t CC_PYTHON_MODULE needed here?
3. Where do the database credentials come from?
4. Migrations run in CC_PRE_RUN_HOOK. What could go wrong with 3 instances?
5. “My app answers Bad Request (400)” — what would you check?

LAB 3 · SAME RUNTIME, TWO MODES

The project decides how Python runs

	Classic mode	Native uv mode
Triggered by	requirements.txt	uv.lock + CC_PYTHON_UV_RUN_COMMAND
Dependencies	pip install	uv sync --locked
HTTP server	Nginx + uWSGI (or gunicorn…)	Your own (uvicorn…)
Entry point	CC_PYTHON_MODULE	CC_PYTHON_UV_RUN_COMMAND
Port	handled by the platform	your server on 0.0.0.0:8080

Support reflex: first find out which mode applies — the same symptom has different causes.

LAB 4 · 30 MIN

Clever Kubernetes Engine

Clever Cloud runs the control plane and heals the VMs.
The platform team runs one or two shared clusters.
Developers deploy in their namespace — with plain kubectl.

In beta · enabled per user · command line only for now (Console support coming)

Three topologies, chosen at creation

Topology	Layout	Typical use
ALL_IN_ONE (default)	Control plane + worker on the same VMs	Dev, test, small teams
DEDICATED_COMPUTE	Control plane VMs + separate node groups	Production
DISTRIBUTED	One VM per control plane component + node groups	Demanding production

Also: flavor, replication factor (1–5), Kubernetes version, node groups, autoscaling (opt-in), persistent storage (Ceph CSI). **Topology and replication factor can’t change later.**

```
clever k8s create --help
clever k8s create {{CLUSTER}} \
--org {{ORG}} --watch
```

LAB 4 · HANDS-ON

One shared cluster, one namespace each — just like at work

1 · CONNECT

```
clever k8s get-kubeconfig \
{{CLUSTER}} --org {{ORG}} \
> ~/.kube/cc-training.yaml
export KUBECONFIG=\
~/.kube/cc-training.yaml
kubectl get nodes
```

2 · DEPLOY

```
kubectl create ns <name>
kubectl apply -n <name> \
-f whoami.yaml
kubectl get pods \
-n <name> -o wide
```

3 · REACH IT

```
kubectl port-forward \
-n <name> \
svc/whoami 8081:80

curl localhost:8081
```

⚠ The kubeconfig gives full access to the cluster: treat it as a secret. Don’t overwrite your own ~/.kube/config.

LAB 4 · DELETE A POD, WATCH WHAT HAPPENS

Who heals what — and who gets the ticket

Kubernetes heals pods

The Deployment recreates a deleted or crashed pod. A pod in CrashLoopBackOff is the user's workload.

```
kubectl logs · get events
```

Clever Cloud heals VMs

A failed control plane or worker VM is replaced automatically. A NotReady node or an unreachable API server is the platform's side.

```
clever k8s activity · get
```

Six things to remember at the next ticket

1. Find the line: platform or user? PaaS app or CKE?
2. Every deployment creates new instances; their disk doesn’t survive.
3. Configuration is env vars — yours, CC_*, the add-ons’. Change → restart.
4. New instances must pass the checks before traffic switches.
5. Read the deployment log first.
6. The project decides the mode (classic vs uv): check which one applies.

WHAT'S NEXT

Next time, you're the one they call

Track	Session
Support L1	User support and qualifying user needs
Support L1	Troubleshooting methodology — break it, fix it
Support L1	Concrete support cases
Support L2	Advanced troubleshooting and monitoring
Support L2	Advanced user support
Support L2	Thematic focus: infrastructure

Before you leave: clean up (Lab 5) — unless we keep resources for the next sessions.